2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)
October 1-5, 2023. Detroit, USA

Consecutive Inertia Drift of Autonomous RC Car via Primitive-based Planning and Data-driven Control

Yiwen Lu[1], Bo Yang[1], Jiayun Li[1], Yihan Zhou[1], Hongshuai Chen[2] and Yilin Mo[1]

***Abstract*— Inertia drift is an aggressive transitional driving maneuver, which is challenging due to the high nonlinearity of the system and the stringent requirement on control and planning performance. This paper presents a solution for the consecutive inertia drift of an autonomous RC car based on primitive-based planning and data-driven control. The planner generates complex paths via the concatenation of path segments called primitives, and the controller eases the burden on feedback by interpolating between multiple real trajectories with different initial conditions into one near-feasible reference trajectory. The proposed strategy is capable of drifting through various paths containing consecutive turns, which is validated in both simulation and reality.**

I. INTRODUCTION

Drifting, a driving technique featuring the intentional loss of traction and the maintenance of large slip-slide angles, has been pursued by both professional human racers and autonomous agents [1]–[4]. Drifting pushes the vehicle to the limit of maneuverability, and hence the study of which may provide insights to full autonomous driving, especially to the precise control of vehicle trajectory when a sudden loss of friction is caused by external conditions like rain or ice [5]. Furthermore, drift maneuvering is faced with limited control authority in a highly unstable region [2] and a stringent requirement on real-time decision making, which are otherwise ubiquitous challenges in the research of agile robots [6]–[8]. Therefore, control and planning techniques developed for drifting can potentially be of general interest in robotics.

Existing research on autonomous drift have mainly focused on either *sustained* or *transient* drift [2], where the former is concerned with the stabilization near an equilibrium of drifting along a circular path [5], [9], [10], while the latter refers to temporary drift maneuvers including drift parking and cornering [2], [11]. A drift manuever well-known among human enthusiasts that fall outside the above two categories is *inertia drift*, which connects a counter-clockwise drifting path with a clockwise one or vice versa within minimal time and distance, and is useful in racing due to its capability to pass through consecutive turns without significant loss of speed. The present work combines sustained and inertia drift to accomplish more complex acrobatics than each of the individual maneuvers, e.g., drifting consecutively along the 8-shaped path illustrated in Fig. 1.

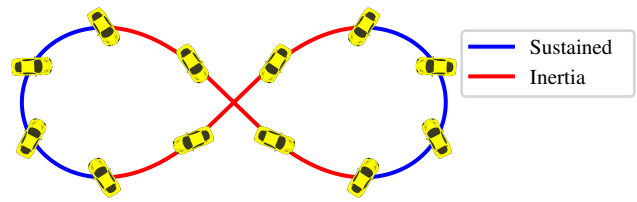


Fig. 1: Illustration of a 8-shaped path combining circular and inertia drift.

The complexity of the drift task considered in this work amounts to various difficulties in control and planning. On one hand, compared to sustained drift along a unidirectional circular path, inertia drift requires traversing through a series of transient states as opposed to the stabilization near a single equilibrium. Consequently, controllers designed for linearized vehicle models around drift equilibria [9] may not apply. In addition, the restricted availability of trajectory data in each particular region of the state space poses challenge to the online system identification that facilitates model-based control [10]. On the other hand, compared to a transient drift, the combination of large sideslip angles at both the beginning and the end of an inertia drift trajectory demands higher controller performance. To be capable of following the path consecutively rather than performing only a one-shot maneuver, the agent also needs a planner that chooses the accurate timings for switching between inertia and sustained drift modes in real time.

The present work addresses the aforementioned challenges through primitive-based planning and data-driven control. By decomposing the drifting path into motion primitives including sustained and inertia drift trajectory segments, the planning problem is reduced to selecting a motion primitive stored offline at each moment based on simple geometry. The primitive-based planner is versatile in the sense that it can be applied to various tasks as long as the desired path comprises known primitives. It is also computationally light compared to sampling- [12] or optimization-based [13] planning methods, and plans onboard at 100Hz. Meanwhile, through the combination of only a handful real trajectories stored offline, the data-driven controller can generate near-feasible reference readily tracked by simple feedback, without relying on explicit system identification. The proposed strategy is verified on both a high-fidelity simulation platform and a real 1/10-scale RC car.

This work is supported by National Natural Science Foundation of China (No. 62192752) and Tsinghua University - Meituan Joint Institute for Digital Life.

[1]Y.Lu, B.Yang, J.Li, Y. Zhou, Y.Mo are with Department of Automation and BNRist, Tsinghua University, Beijing, China: {luyw20, yang-b21, lijiayun22, zhou-yh19} @mails.tsinghua.edu.cn, ylmo@tsinghua.edu.cn.

[2] H.Chen is with the Department of Autonomous Vehicles, Meituan, Beijing, China: chenhongshuai@meituan.com.

2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) | 978-1-6654-9190-7/23/$31.00 ©2023 IEEE | DOI: 10.1109/IROS55552.2023.10341461

Authorized licensed use limited to: Universitatsbibliothek Erlangen Nurnberg. Downloaded on April 09,2026 at 17:50:38 UTC from IEEE Xplore. Restrictions apply.

The contributions of this work can be summarized as follows:

1) This work achieves the autonomous control of *inertia drift*, a challenging maneuver that has not been explicitly studied in the literature.
2) Various complex maneuvering tasks unaccomplished by previous methods [3], [14], including drifting along an 8-shaped path, a three-circle path, and an "Olympic Rings" path (refer to Fig. 8) within a confined space, are tackled by the proposed method in a unified manner.
3) The proposed method is physically verified by an RC car with onboard planning and control (refer to video attachment), demonstrating the promise of aggressive real-world maneuvering.

An extended review of the literature can be found in [15].

II. PRELIMINARIES

A. Target platform

The target platform of the proposed strategy is a 1/10-scale, four-wheel-driven RC car with locked differential shown in Fig. 2, inspired by the F1TENTH [16] project. The car is equipped with an NVIDIA Jetson TX2 single board computer which runs the planner and controller onboard, and controls the electric motor of the car through a VESC motor controller. High-accuracy pose information is provided by a motion capture system, based on which the (angular) velocities and accelerations are estimated using a Kalman filter.

Fig. 2: The RC car platform

For strategy design and verification, a self-developed mixed-fidelity simulation platform is used. The platform is implemented using the `DifferentialEquations` package in Julia, and supports both a bicycle model with Pacejka tire [12], which is used for strategy design, and a higher-fidelity four-wheel model [17] with actuator delay, which is used for strategy verification.

B. Modeling

The proposed strategy is based on the bicycle model similar to the one used in [5], [18], illustrated in Fig. 3. The state and input variables of this bicycle model are:

$$\mathbf{X} = (x, y, \psi, \dot{x}, \dot{y}, \dot{\psi}), \mathbf{U} = (\delta, \omega), \tag{1}$$

where x, y are the coordinate of the center of mass of the car in the world frame, ψ is the yaw angle, δ is the front wheel steering angle, and ω is the rotational speed of both wheels.

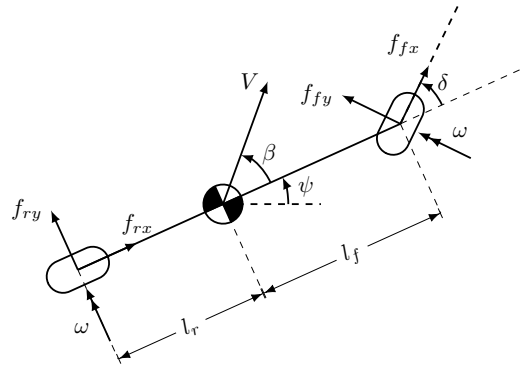


Fig. 3: Illustration of bicycle model

Consider the following variables that describe the motion of the car:

$$r = \dot{\psi} \qquad \text{(yaw rate)}, \tag{2}$$

$$\beta = \operatorname{atan2}(\dot{y}, \dot{x}) - \psi \qquad \text{(sideslip angle)}, \tag{3}$$

$$V = \sqrt{\dot{x}^2 + \dot{y}^2} \qquad \text{(speed)}. \tag{4}$$

Assuming that $V > 0$, similarly to [3], one can derive a reduced-order model w.r.t. (r, β, V) as follows:

$$\begin{aligned} \dot{r} &= \frac{l_f\left(f_{fy}c_\delta + f_{fx}s_\delta\right) - l_r f_{ry}}{I_z}, \\ \dot{\beta} &= \frac{f_{fy}c_{\delta-\beta} + f_{fx}s_{\delta-\beta} + f_{ry}c_\beta - f_{rx}s_\beta}{mV} - r, \\ \dot{V} &= \frac{-f_{fy}s_{\delta-\beta} + f_{fx}c_{\delta-\beta} + f_{ry}s_\beta + f_{rx}c_\beta}{m}, \end{aligned} \tag{5}$$

where $c., s.$ stand for $\cos(\cdot), \sin(\cdot)$ respectively, m is the mass of the car, I_z is the moment of inertia of the car about the vertical axis, l_f, l_r are the distances from the center of mass to the front and rear axles respectively, and $f_{fx}, f_{fy}, f_{rx}, f_{ry}$ are the components of friction forces as exemplified in Fig. 3. The input of this reduced-order model is $(\delta, f_{fx}, f_{fy}, f_{rx}, f_{ry})$, which can be derived from the actual input (ω, δ) through the Pacejka tire model.

It is worth noticing that given (r, β, V), the motion of the pose (x, y, ψ) can be determined by the following purely kinematic equation:

$$\dot{x} = V\cos(\beta + \psi),\ \dot{y} = V\sin(\beta + \psi),\ \dot{\psi} = r, \tag{6}$$

which legitimates (5) as a rotation- and translation-invariant description of the motion of the car, and lays the foundation for the definition of motion primitive. We shall refer to (r, β, V) as a "reduced state" hereafter.

C. Problem statement

The aim of this work is to use steering and speed commands (which correspond to the control inputs δ, ω in (1)) adhering to actuation limits to perform autonomous maneuvers that interleave sustained drift with inertia drift, within a confined space. The two types of drift are both characterized by a nontrivial sideslip angle β (c.f. (3)), which are described respectively as follows:

- Sustained drift: maintaining an approximately constant sideslip angle, e.g., $\beta \approx 1\,\text{rad}$ for a clockwise path or

Authorized licensed use limited to: Universitatsbibliothek Erlangen Nurnberg. Downloaded on April 09,2026 at 17:50:38 UTC from IEEE Xplore. Restrictions apply.

$\beta \approx -1\,\mathrm{rad}$ for a counterclockwise path, while traveling along a prescribed circle. Let the desired sideslip angle be β_{ref}, and the prescribed circle be centered at (x_c, y_c) with radius R, then we define the error of sustained drift to be

$$\begin{aligned} &e = (e_{\mathrm{slip}}, e_{\mathrm{pos}}, e_{\mathrm{dir}}), \text{ where} \\ &e_{\mathrm{slip}} = \beta - \beta_{\mathrm{ref}}, \\ &e_{\mathrm{pos}} = \sqrt{(x - x_c)^2 + (y - y_c)^2} - R, \\ &e_{\mathrm{dir}} = \psi + \beta - \mathrm{atan2}(y_c - y, x_c - x) + \frac{\mathrm{sgn}(\beta)\pi}{2}. \end{aligned} \tag{7}$$

The definitions of the position error e_{pos} and the direction error e_{dir} are illustrated in Fig. 4.

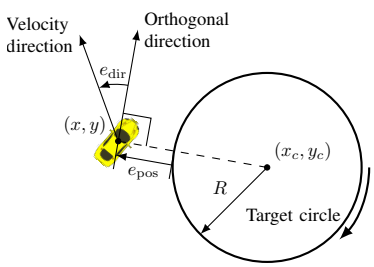


Fig. 4: Illustration of the error terms e_{pos} and e_{dir}: e_{pos} is the signed distance between the car and the target circle, and e_{dir} is the difference between the velocity direction $\psi+\beta$ and the desired direction $\mathrm{atan2}(y_c - y, x_c - x) - \mathrm{sgn}(\beta)(\pi/2)$, which is orthogonal to the line passing the car and the center of the target circle.

- Inertia drift: rapidly altering the sign of the sideslip angle, e.g., from $\beta \approx -1\,\mathrm{rad}$ to $\beta \approx 1\,\mathrm{rad}$ or vice versa, while traveling along an S-shaped path. This is the transitional maneuver between two stages of sustained drift in opposite directions. Therefore, instead of defining a dedicated error function for inertia drift, we consider an inertia drift maneuver to be successful if it creates a desirable initial condition for the subsequent sustained drift maneuver.

The overall objective is to track a path defined by multiple target circles, using sustained drift when traveling along each circle and inertia drift when moving between circles, with the error defined in (7) bounded across different stages of sustained drift. Instances of target paths include the 8-shaped path shown in Fig. 1 and other more complex paths comprising three or more circles show in Fig. 8. These are common training tasks for drift enthusiasts, and can therefore serve as benchmarks of autonomous car maneuvering in challenging scenarios.

III. METHOD

This section is intended to resolve:

1) *Planning*: How to select the timings of switching between sustained and inertia drift according to the task specification?
2) *Control*: How to track sustained and inertia drift paths respectively?

For the control of sustained drift, curvature feedback for controlling the path radius, along with a circumnavigation rule [19] for stabilizing the path center, has proved effective and robust [5], and is therefore adopted for this work. Interested readers are referred to [5] for the technical details. The remainder of this section is dedicated to the high-level planning and the control of inertia drift.

An overview of the proposed strategy is shown in Fig. 5. The primitive-based planner selects the timings of switching by referring to an offline-generated library of inertia drift primitives. Once it decides to enter inertia drift, it passes a subset of the library, known as candidate primitives, to the data-driven controller. The latter determines a near-feasible trajectory by taking the weighted average of the candidate primitives utilizing a pre-computed lookup table, and tracks the near-feasible trajectory with a feedforward / feedback controller. The details of planning and control are described respectively below.

A. Primitive-based planning

To keep complex maneuvers tractable, one can decompose a task into relatively simple motion primitives. In particular, we focus on primitives of drift. Formally, a drift primitive can be defined as follows:

Definition 1. *A drift primitive is defined as the following 5-tuple:*

$$\begin{aligned} &\mathcal{P} = (T, \tau, \Delta x_b, \Delta y_b, \Delta\psi)\,, \\ &\textit{with } \tau = \{(r(t), \beta(t), V(t), \omega(t), \delta(t))\}_{t=1}^{T}\,, \end{aligned} \tag{8}$$

where $T \in \mathbb{N}^*$ is the number of time steps, τ is a trajectory of the quantities $r, \beta, V, \omega, \delta$ defined in (1) *and* (5) *sampled over time, $\Delta x_b, \Delta y_b$ are the difference between terminal and initial x, y coordinates as measured in the initial body frame, and $\Delta\psi$ is the difference between terminal and initial yaw angles.*

Note that the representation of drift primitive in (8) may seem redundant, as the quantities $\Delta x_b, \Delta y_b, \Delta\psi$ may all be inferred from the trajectory τ. However, as it will become clear later, the storage of these quantities allows real-time planning without the need for access to the entire trajectory. Therefore, it is an essential feature of the proposed primitive-based planning method.

Complex drift maneuvers can be achieved through the concatenation of sustained and inertia drift primitives. The rest of this subsection is devoted to explaining how to generate inertia drift primitives offline and how to perform the concatenation online.

1) Generation of inertia drift primitives: Generating an inertia drift primitive is in essence finding a dynamically feasible path from an initial reduced state (r_0, β_0, V_0) to a desired terminal reduced state $(r_{\mathrm{des}}, \beta_{\mathrm{des}}, V_{\mathrm{des}})$, where r_0 (resp. β_0) and r_{des} (resp. β_{des}) have opposite signs. Therefore, given (r_0, β_0, V_0) and $(r_{\mathrm{des}}, \beta_{\mathrm{des}}, V_{\mathrm{des}})$, a feasible primitive for the bicycle model can be generated by solving the following trajectory optimization problem and sampling

Authorized licensed use limited to: Universitatsbibliothek Erlangen Nurnberg. Downloaded on April 09,2026 at 17:50:38 UTC from IEEE Xplore. Restrictions apply.

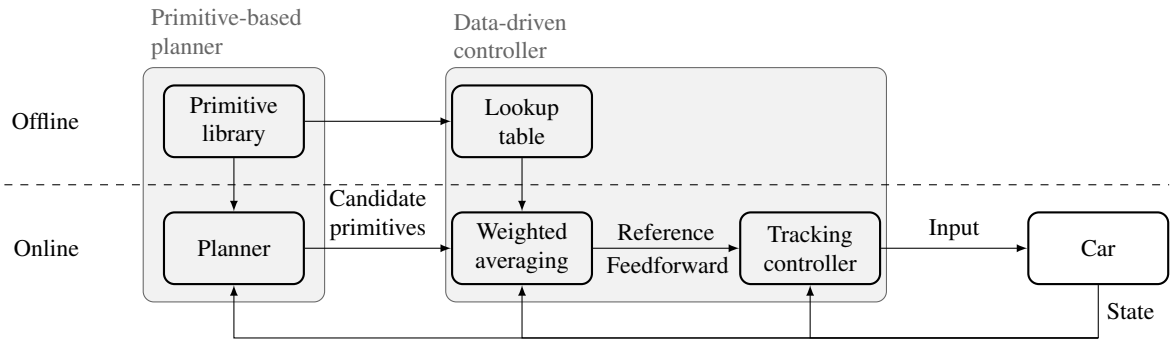


Fig. 5: Overview of the proposed planning and control strategy

the resulting trajectory:

$$\begin{aligned}
&\min_{t_f, r, \beta, V, \omega, \delta} \; J = \int_0^{t_f} (r_\omega \omega(t)^2 + r_\delta \delta(t)^2) \mathrm{d}t, \\
&\text{s.t. } r(0) = r_0, \beta(0) = \beta_0, V(0) = V_0, \\
&\quad \|(r(t_f), \beta(t_f), V(t_f)) - (r_{\text{des}}, \beta_{\text{des}}, V_{\text{des}})\| \le \epsilon, \qquad (9) \\
&\quad 0 \le \omega(t) \le \omega_{\max}, -\delta_{\max} \le \delta(t) \le \delta_{\max}, \forall t \in [0, t_f], \\
&\quad \text{and dynamics constraint (5),}
\end{aligned}$$

where r_ω, r_δ are weight coefficients for penalizing large inputs, $\epsilon > 0$ a tolerance threshold for inexact terminal state, $\omega_{\max}, \delta_{\max}$ are the actuation limits, and the forces $f_{fx}, f_{fy}, f_{rx}, f_{ry}$ in the dynamics constraint (5) are posed as functions of the inputs $(\omega(t), \delta(t))$ via the Pacejka tire model. With the total time t_f fixed, the above trajectory optimization problem can be solved by iLQR, and the optimal t_f can be search via bisection.

The primitive obtained from solving (9), however, may be infeasible for the target platform, i.e., either high-fidelity simulation or real car, due to modeling errors caused by the simplifications in the bicycle model and parameter mismatch. Therefore, during offline calibration, each primitive generated using the bicycle model is fed to the target platform as a reference trajectory and tracked by a feedforward / feedback controller. The resulting *real* trajectory, once passing a check that the terminal $(r(t_f), \beta(t_f), V(t_f))$ is not too far from the desired $(r_{\text{des}}, \beta_{\text{des}}, V_{\text{des}})$, is stored into a primitive library for the online planner and controller to query.

2) Planner logic: The goal of online planning is concatenating motion primitives with appropriate timings, specifically interleaving sustained drift with inertia drift. The logic of the planner can be represented by the finite state machine in Fig. 6. The conditions that cause the planner to switch from one primitive to another are explained as follows:

a) From sustained drift to inertia drift: Assume that the current pose of the car is (x, y, ψ), the current reduced state is (r, β, V), and the error defined in (7) against the current target circle for sustained drift is smaller than a threshold. For a particular drift primitive $\mathcal{P}$ as specified in (8), the planner attempts to "place" it in the world frame applying

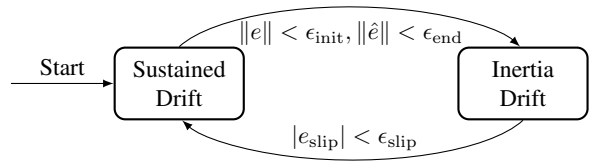


Fig. 6: Planner logic: e is the current error against the current target circle; $\hat{e}$ is the predicted terminal error against the next target circle if inertia drift is applied now; e_{slip} is the error in the sideslip angle; $\epsilon_{\text{init}}, \epsilon_{\text{end}}, \epsilon_{\text{slip}}$ are design constants for error tolerance. Refer to (7) and Fig. 4 for the definition of error.

$(\Delta x_b, \Delta y_b, \Delta\psi)$ to (x, y, ψ) as follows:

$$\begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = \begin{bmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{bmatrix} \begin{bmatrix} \Delta x_b \\ \Delta y_b \end{bmatrix},$$
$$(\hat{x}, \hat{y}, \hat{\psi}) = (x + \Delta x, y + \Delta y, \psi + \Delta\psi),$$

where $(\hat{x}, \hat{y}, \hat{\psi})$ stands for the predicted terminal pose if the planner switches to $\mathcal{P}$ immediately. If $(\hat{x}, \hat{y}, \hat{\psi})$, along with $(r(T), \beta(T), V(T))$ stored in $\mathcal{P}$, fits the target circular path from the task specification, then it can be said it is appropriate timing to switch to $\mathcal{P}$. In particular, the "fitness" to a target circle can be measured by the error defined in (7), and switching happens when the error is small enough.

So far, it remains undiscussed how to select a primitive $\mathcal{P}$ from the primitive library. Considering the actual initial reduced state (r, β, V) may not match $(r(1), \beta(1), V(1))$ of any primitive from the pre-stored library, it is necessary to interpolate from multiple primitives. Such interpolation is closely related to data-driven control, and the detailed discussion of that aspect shall be deferred to Subsection III-B.

b) From inertia drift to sustained drift: As revealed by previous research [5], [9], a key to successfully maintaining sustained drift is a large initial sideslip angle β. Therefore, as shown in Fig. 6, sustained drift can take over from inertial drift when β is close to the target value of the next target circle.

B. Data-driven control

Data-driven control is concerned with generating and tracking near-feasible trajectories under particular initial conditions using collected data. This subsection is devoted to

Authorized licensed use limited to: Universitatsbibliothek Erlangen Nurnberg. Downloaded on April 09,2026 at 17:50:38 UTC from IEEE Xplore. Restrictions apply.

developing the data-driven control technique for inertia drift primitives.

1) Posing data-driven control as a weighted average of primitives: The goal of data-driven control in this work is to find a near-feasible primitive for the current initial reduced state (r, β, V), given multiple candidate primitives with different initial reduced states from the library. Let $\mathcal{P}_1, \ldots, \mathcal{P}_n$ be n candidate primitives. Intuitively, a near-feasible primitive starting from (r, β, V) can be obtained by the weighted average of $\mathcal{P}_1, \ldots, \mathcal{P}_n$, with the weight vector $\lambda \in \mathbb{R}^n$ determined by:

$$\min \|\lambda\|_2^2 \tag{10}$$

$$\text{s.t.} \sum_{i=1}^{n} \lambda_i \cdot (r_i(1), \beta_i(1), V_i(1)) = (r, \beta, V), \tag{11}$$

$$\sum_{i=1}^{n} \lambda_i = 1,\ \lambda_i \geq 0, i = 1, \ldots, n, \tag{12}$$

where $(r_i(i), \beta_i(1), V_i(1))$ is $(r(1), \beta(1), V(1))$ from $\mathcal{P}_i$, constraint (11) makes sure the initial reduced state of the new primitive matches (r_0, β_0, V_0), and constraint (12) guarantees the weights are normalized and nonnegative. The objective (10), i.e., minimum 2-norm, promotes an even distribution of weights. This is based on the intuition that primitive data collected from a real car may be corrupted by random noise, which can be alleviated by averaging multiple trajectories.

It is assumed that the optimization problem (10)-(12) is feasible, i.e., (r, β, V) belongs to the convex hull spanned by $\{(r_i(1), \beta_i(1), V_i(1))\}_{i=1}^n$, which can be ensured when the candidate primitives have sufficiently diverse initial reduced states.

2) Weighted average of primitives via soft dynamic time warping: Given the vector λ determined by (10)-(12), defining the average of $\mathcal{P}_1, \ldots, \mathcal{P}_n$ weighted by λ is still highly nontrivial: by (8), each primitive $\mathcal{P}_i$ contains a trajectory $\tau_i = \{(r_i(t), \beta_i(t), V_i(t), \omega_i(t), \delta_i(t))\}_{t=1}^{T_i}$, whose length T_i may vary across primitives, and therefore naive methods for weighted averaging do not apply. To address the aforementioned problem, one can cast the weighted averaging of trajectories $\tau_1, \ldots, \tau_n$ with weight vector λ into the following optimization problem:

$$\min_{\bar{\tau}} \sum_{i=1}^{n} \lambda_i d(\bar{\tau}, \tau_i), \tag{13}$$

where d is a generalized distance metric function between trajectories. In particular, we choose d to be the dtw_γ metric [20], a log-sum-exp approximation of the minimum distance among all possible alignments between two temporal sequences that may vary in length and speed. We choose dtw_γ over other metrics for sequences [21] because it is differentiable (where the parameter γ controls the smoothness), and therefore allows for finding a local minimum of (13) using gradient-based optimizers. Based on the above discussions, the weighted average of primitives can be defined as follows:

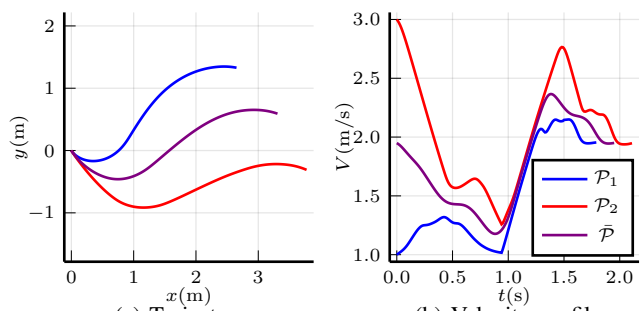
Definition 2. *For drift primitives $\mathcal{P}_i = (T_i, \tau_i, \Delta x_{bi}, \Delta y_{bi}, \Delta\psi_i)$ $(i = 1, \ldots, n)$, a weight vector $\lambda \in \mathbb{R}^n$ satisfying* (12)*, and a scalar $\gamma > 0$, the weighted average of $\mathcal{P}_1, \ldots, \mathcal{P}_n$ weighted by λ with smoothness γ is defined as:*

$$\bar{\mathcal{P}} = (\bar{T}, \bar{\tau}, \overline{\Delta x_b}, \overline{\Delta y_b}, \overline{\Delta\psi}), \tag{14}$$

where $\bar{T} = \sum_{i=1}^n \lambda_i T_i$, $\bar{\tau}$ is the solution of (13) *with $d = \mathrm{dtw}_\gamma$ defined in [20], $\overline{\Delta x_b} = \sum_{i=1}^n \lambda_i \Delta x_{bi}$, $\overline{\Delta y_b} = \sum_{i=1}^n \lambda_i \Delta y_{bi}$ and $\overline{\Delta\psi} = \sum_{i=1}^n \lambda_i \Delta\psi$.*

Fig. 7 shows the weighted average $\bar{\mathcal{P}}$ of two primitives $\mathcal{P}_1, \mathcal{P}_2$ with different initial velocities, with $\lambda_1 = \lambda_2 = 0.5$ and $\gamma = 50$. It can be observed that both the velocity profile and the trajectory agree with the intuition of averaging.

As an addition note, since solving (13) is time-consuming and can hardly be completed in real time, it is pre-computed offline and the solutions are stored in a lookup table. This table maps an initial reduced states (r, β, V), which are sampled regularly from a fine grid, to corresponding near-feasible trajectories $\bar{\tau}$ generated by solving the optimization problems (10)-(12) and (13). In online planning and control, the average trajectory $\bar{\tau}$ in (14) is obtained by looking up for the nearest neighbor of the current reduced state from the table.



(a) Trajectory (b) Velocity profile

Fig. 7: Demonstration of averaging drift primitives via the dtw_γ metric: purple trajectory is the average of red and blue ones.

3) Online tracking controller: Given a near-feasible drift primitive $\bar{\mathcal{P}}$ for the current initial reduced state, constructed by weighted averaging multiple feasible primitives with various initial reduced states according to Definition 2, a simple feedforward / feedback controller can be designed to track the primitive online. The controller is based on the intuition that increasing the wheel speed ω causes both the speed V and the sideslip angle β to increase, and that increasing the steering angle δ causes the yaw rate r to increase in the corresponding direction. Specifically:

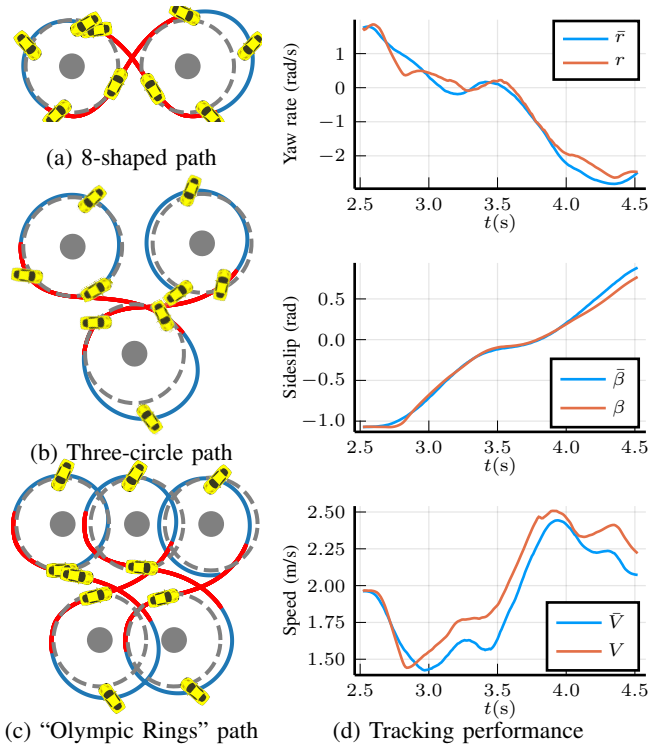
$$\omega_t = \bar{\omega}_t - k_V(V_t - \bar{V}_t) - k_\beta(|\beta_t| - |\bar{\beta}_t|),$$
$$\delta_t = \bar{\delta}_t - k_r(r_t - \bar{r}_t),$$

for any time step t, where $(\bar{r}_t, \bar{\beta}_t, \bar{V}_t, \bar{\omega}_t, \bar{\delta}_t)$ are available from the trajectory of $\bar{\mathcal{P}}$, and k_r, k_β, k_V are controller parameters.

Authorized licensed use limited to: Universitatsbibliothek Erlangen Nurnberg. Downloaded on April 09,2026 at 17:50:38 UTC from IEEE Xplore. Restrictions apply.

IV. RESULTS

The performance of the proposed strategy on the high-fidelity simulation platform is shown in Fig. 8. One can observe that the data-driven controller is capable of generating near-feasible reference and delivers a decent tracking performance, and that the primitive-based planner adapts to various tasks through the concatenation of a fixed set of primitives.



(a) 8-shaped path

(b) Three-circle path

(c) "Olympic Rings" path

(d) Tracking performance

Fig. 8: Simulation result in high-fidelity platform. (a)(b)(c): paths for various tasks (blue for sustained drift, red for inertia drift, gray for target circles); (d): reference and actual reduced state of a single drift primitive ($\bar{r}, \bar{\beta}, \bar{V}$ are available from the reference primitive $\bar{\mathcal{P}}$).

Snapshots of the hardware experiment are shown in Fig. 9, from which one can observe that the proposed strategy maintains consistent performance across simulation and reality. The complete experiment results are included in the accompanying video. Readers are referred to [15] for the detailed experimental setup.

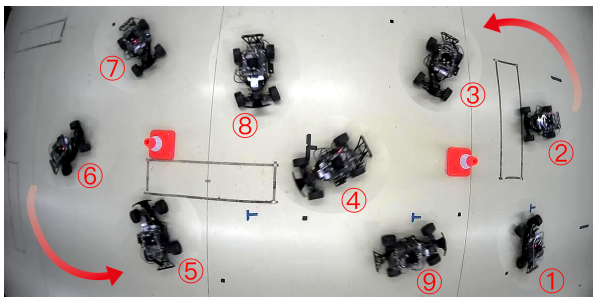


Fig. 9: Experiment result for 8-shaped path (9 frames of video stacked together; red digits indicate order of frames)

REFERENCES

[1] M. Cutler and J. P. How, "Autonomous drifting using simulation-aided reinforcement learning," in *2016 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, May 2016, pp. 5442–5448.

[2] F. Zhang, J. Gonzales, S. E. Li, F. Borrelli, and K. Li, "Drift control for cornering maneuver of autonomous vehicles," *Mechatronics*, vol. 54, pp. 167–174, Oct. 2018.

[3] J. Y. Goh, T. Goel, and J. Christian Gerdes, "Toward Automated Vehicle Control Beyond the Stability Limits: Drifting Along a General Path," *Journal of Dynamic Systems, Measurement, and Control*, vol. 142, no. 2, p. 021004, Feb. 2020.

[4] P. Cai, X. Mei, L. Tai, Y. Sun, and M. Liu, "High-speed autonomous drifting with deep reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1247–1254, 2020.

[5] B. Yang, Y. Lu, X. Yang, and Y. Mo, "A hierarchical control framework for drift maneuvering of autonomous vehicles," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 1387–1393.

[6] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.

[7] X. B. Peng, E. Coumans, T. Zhang, T.-W. Lee, J. Tan, and S. Levine, "Learning agile robotic locomotion skills by imitating animals," *arXiv preprint arXiv:2004.00784*, 2020.

[8] J. M. Levin, M. Nahon, and A. A. Paranjape, "Real-time motion planning with a fixed-wing uav using an agile maneuver space," *Autonomous Robots*, vol. 43, no. 8, pp. 2111–2130, 2019.

[9] M. Baur and L. Bascetta, "An experimentally validated lqr approach to autonomous drifting stabilization," in *2019 18th European Control Conference (ECC)*. IEEE, 2019, pp. 732–737.

[10] E. Joa, H. Cha, Y. Hyun, Y. Koh, K. Yi, and J. Park, "A new control approach for automated drifting in consideration of the driving characteristics of an expert human driver," *Control Engineering Practice*, vol. 96, p. 104293, 2020.

[11] E. Jelavic, J. Gonzales, and F. Borrelli, "Autonomous Drift Parking using a Switched Control Strategy with Onboard Sensors," *IFAC-PapersOnLine*, vol. 50, no. 1, pp. 3714–3719, July 2017.

[12] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *The international journal of robotics research*, vol. 30, no. 7, pp. 846–894, 2011.

[13] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, "Stomp: Stochastic trajectory optimization for motion planning," in *2011 IEEE international conference on robotics and automation*. IEEE, 2011, pp. 4569–4574.

[14] F. Domberg, C. C. Wembers, H. Patel, and G. Schildbach, "Deep Drifting: Autonomous Drifting of Arbitrary Trajectories using Deep Reinforcement Learning," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, May 2022, pp. 7753–7759.

[15] Y. Lu, B. Yang, J. Li, Y. Zhou, H. Chen, and Y. Mo, "Consecutive inertia drift of autonomous rc car via primitive-based planning and data-driven control," *arXiv preprint arXiv:2306.12604*, 2023.

[16] M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1tenth: An open-source evaluation environment for continuous control and reinforcement learning," *Proceedings of Machine Learning Research*, vol. 123, 2020.

[17] S. Milani, H. Marzbani, and R. N. Jazar, "Vehicle drifting dynamics: discovery of new equilibria," *Vehicle system dynamics*, vol. 60, no. 6, pp. 1933–1958, 2022.

[18] Y. Lu, B. Yang, and Y. Mo, "Two-timescale Mechanism-and-Data-Driven Control for Aggressive Driving of Autonomous Cars," in *2021 China Automation Congress (CAC)*. IEEE, 2021, pp. 7874–7879.

[19] F. Dong, K. You, and S. Song, "Target encirclement with any smooth pattern using range-based measurements," *Automatica*, vol. 116, p. 108932, 2020.

[20] M. Cuturi and M. Blondel, "Soft-dtw: a differentiable loss function for time-series," in *International conference on machine learning*. PMLR, 2017, pp. 894–903.

[21] M. Cuturi, J.-P. Vert, O. Birkenes, and T. Matsui, "A kernel for time series based on global alignments," in *2007 IEEE International Conference on Acoustics, Speech and Signal Processing-ICASSP'07*, vol. 2. IEEE, 2007, pp. II–413.

Authorized licensed use limited to: Universitatsbibliothek Erlangen Nurnberg. Downloaded on April 09,2026 at 17:50:38 UTC from IEEE Xplore. Restrictions apply.